



TousLocation

Application de gestion de location de matériel

Laravel · React · MySQL

Projet de Fin d'Études

Réalisé par : _____

Encadré par : _____

Filière : _____

Établissement : _____

Année universitaire : 2025 – 2026

Résumé

Ce projet consiste en la conception et la réalisation de **TousLocation**, une application web de **gestion de location de matériel** destinée aux entreprises marocaines. Bâtie sur une architecture découplée **Laravel (API REST) + React (SPA)** avec une base **MySQL**, elle couvre l'ensemble du cycle d'exploitation : gestion du catalogue et du stock, locations (avec disponibilité en temps réel, TVA et paiements partiels), approvisionnement (achats), ventes, dépenses, ajustements d'inventaire et **reporting financier** (bénéfice et locations). L'application est **multi-tenant** (chaque entreprise dispose d'un espace isolé), **sécurisée** (Laravel Sanctum, contrôle d'accès par rôle), **responsive** et **bilingue français / arabe (RTL)**. Elle est adaptée au contexte marocain (Dirham, TVA, format des numéros et des dates).

Mots-clés : location de matériel, Laravel, React, API REST, multi-tenant, gestion de stock, TVA, facturation PDF, Maroc.

Table des matières

1. Introduction
 2. Contexte et problématique
 3. Objectifs du projet
 4. Fonctionnalités principales
 5. Stack technique
 6. Architecture de l'application (MVC)
 7. Modèle de données
 8. Architecture multi-tenant et sécurité
 9. Interface utilisateur
 10. Description détaillée des modules
 11. Adaptation au contexte marocain
 12. Déploiement et installation
 13. Comptes de démonstration
 14. Perspectives d'évolution
 15. Conclusion
-

1. Introduction

TousLocation est une application web complète destinée aux entreprises de **location de matériel** (informatique, audiovisuel, événementiel, chantier...). Elle remplace les méthodes manuelles (papier, Excel) par une plateforme centralisée, sécurisée et multi-entreprises, couvrant l'ensemble du cycle de vie du matériel : **achat** → **stock** → **location** → **vente** → **dépenses** → **reporting**.

L'application est construite sur une architecture **découplée** : une API REST **Laravel** côté serveur et une interface **React** (Single Page Application) côté client, le tout adossé à une base **MySQL / MariaDB**.

2. Contexte et problématique

La gestion traditionnelle de la location de matériel souffre de plusieurs limites :

- Difficulté de suivi des locations en cours et de la disponibilité réelle ;
- Risque élevé d'erreurs humaines et de double réservation ;
- Lenteur dans la création des contrats et des factures ;
- Absence de vision financière consolidée (revenus, coûts, bénéfice) ;
- Faible sécurité et aucune séparation des données entre entités.

TousLocation répond à ces problèmes par l'automatisation, le contrôle de disponibilité en temps réel, la génération automatique de documents et un reporting financier intégré.

3. Objectifs du projet

- Gérer les **utilisateurs** et les rôles (super-admin, manager, employé, client).
- Gérer le **catalogue de matériel** avec images, disponibilité et calendrier.

- Gérer le **cycle de location** (réservation, paiements partiels, retour, contrat PDF).
- Gérer l'**approvisionnement** (achats fournisseurs) et les **ventes**.
- Suivre les **dépenses** et les **ajustements de stock** valorisés.
- Fournir des **rapports** (bénéfice, locations).
- Garantir l'**isolation des données** entre entreprises (multi-tenant).
- Offrir une interface **moderne, responsive et bilingue (français / arabe)**.

4. Fonctionnalités principales

Domaine	Fonctionnalités
Utilisateurs	Authentification (Sanctum), rôles, gestion des managers par le super-admin
Matériel	CRUD, images, catégories, marques, unités, devises, calendrier de disponibilité, battement post-location
Locations	Création multi-articles, tarification au jour, TVA, paiements partiels, statuts, contrat / facture PDF
Tableau de bord	KPI + tableau Kanban des locations (glisser-déposer pour changer le statut)
Approvisionnement	Fournisseurs, achats (statut <i>en attente</i> / <i>reçu</i>) alimentant le stock
Ventes	Vente de matériel avec TVA, décrémentant le stock
Dépenses	Charges (loyer, salaires, électricité...) par catégorie
Ajustements	Entrées / sorties de stock avec motif et valorisation au prix d'achat
Rapports	Rapport de bénéfice et rapport de locations par période
Paramètres	TVA, devises, unités, catégories, marques, types de paiement (onglets)
Transverse	Multi-tenant, multi-devises, FR/AR (RTL), notifications (toasts), responsive

5. Stack technique

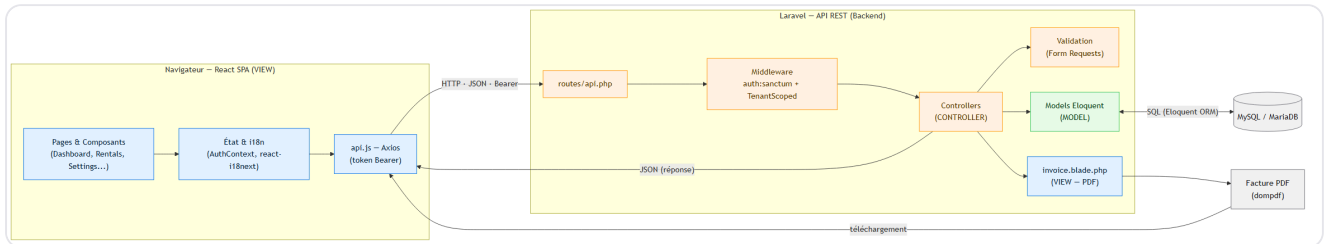
Couche	Technologie	Rôle
Frontend	React 18 + Vite	Interface SPA
Routing	React Router	Navigation
HTTP	Axios	Appels API (token Bearer)
i18n	react-i18next	Français / Arabe (RTL)
Icônes	lucide-react	Iconographie
Backend	Laravel (PHP 8.3)	API REST, logique métier
Auth	Laravel Sanctum	Jetons d'API
PDF	barryvdh/laravel-dompdf	Factures / contrats

Base de données	MySQL / MariaDB	Persistence
Outils	Composer, npm, Git	Build & dépendances

6. Architecture de l'application (MVC)

L'application suit le patron **Modèle-Vue-Contrôleur** côté Laravel, complétée par une **Vue React** déconnectée :

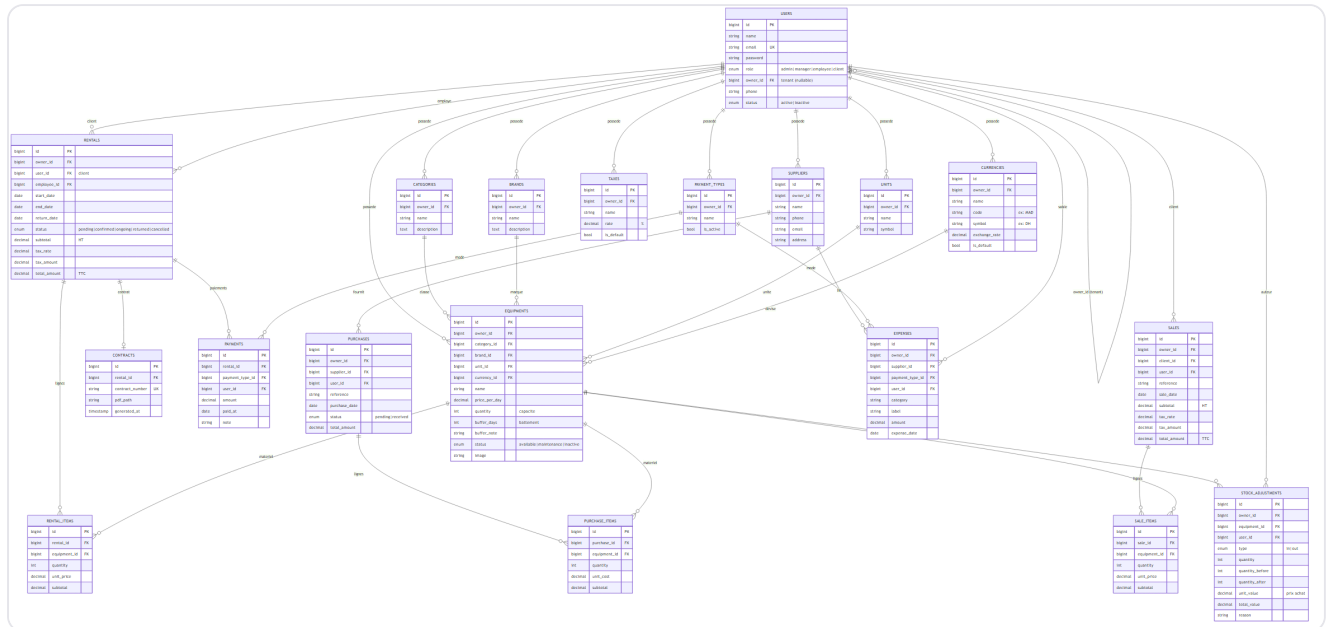
- **Modèle (Model)** : modèles Eloquent (`Equipment` , `Rental` , `Sale` , `Purchase` ...) et l'ORM gérant la base MySQL.
- **Vue (View)** : l'interface **React** (pages, composants) et le gabarit **Blade** pour la génération PDF des factures.
- **Contrôleur (Controller)** : les contrôleurs Laravel, protégés par des middlewares (`auth:sanctum` , scoping multi-tenant) et la validation par *Form Requests*.



Flux d'une requête : React (Axios) → `routes/api.php` → middleware (authentification + isolation tenant) → contrôleur → validation → modèle Eloquent → MySQL → réponse JSON → React. La génération de facture passe par un gabarit Blade rendu en PDF via dompdf.

7. Modèle de données

La base comprend **19 tables** organisées en référentiels, inventaire, locations, achats, ventes, dépenses et ajustements. La quasi-totalité des tables porte une colonne `owner_id` assurant l'isolation multi-tenant.



Principes clés :

- Le **matériel** (`equipments`) référence une catégorie, une marque, une unité et une devise. `quantity` représente la **capacité totale**.
- Une **location** (`rentals`) possède des lignes (`rental_items`), au plus un **contrat** (`contracts`) et plusieurs **paiements** (`payments`).
- Un **achat** reçu alimente le stock ; une **vente** le décrémente ; un **ajustement** le corrige avec une valorisation (`unit_value` $\approx$ dernier prix d'achat).
- La **TVA** appliquée est figée par document (`tax_rate` , `tax_amount`).

Diagramme interactif et export PNG : [database-schema.html](#) .

8. Architecture multi-tenant et sécurité

L'application est **multi-tenant** : chaque **manager** dispose d'un espace de travail totalement isolé (matériel, clients, locations, ventes, achats, dépenses, paramètres).

- Super-admin** : voit et gère toutes les données de tous les espaces ; crée les comptes managers.
- Manager** : ne voit que ses propres données (`owner_id` = son identifiant).
- Employé / Client** : rattachés à un manager via `owner_id` .

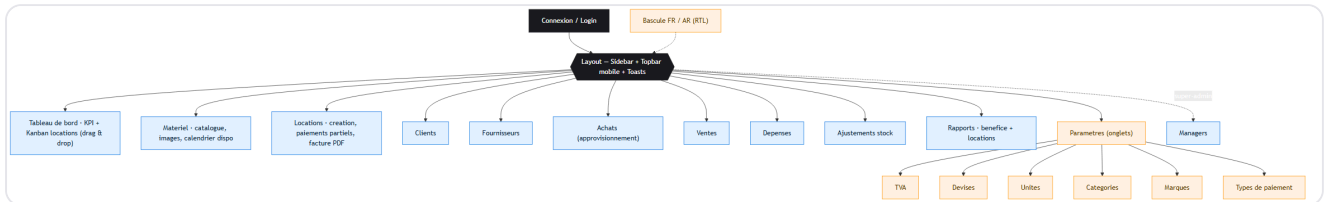
Mécanismes de sécurité :

- Authentification par **jeton Bearer (Sanctum)**, mots de passe **hachés (bcrypt)**.
- Trait TenantScoped** appliqué à tous les contrôleurs : filtrage automatique par tenant et vérification d'appartenance (réponse **404** en cas d'accès croisé).

- **Validation serveur** systématique (Form Requests).
 - Contrôle d'accès par rôle (ex. seul le super-admin accède à la gestion des managers).
-

9. Interface utilisateur

L'interface adopte un design épuré inspiré d'Apple : barre latérale translucide, typographie SF, surfaces arrondies, couleurs système. Elle est **responsive** (menu en tiroir sur mobile), **bilingue FR/AR avec sens RTL**, et utilise des **notifications toast** ainsi qu'une **animation de chargement**.



Points forts UX :

- **Tableau Kanban** des locations en page d'accueil (glisser-déposer pour confirmer / retourner / annuler).
- **Calendrier de disponibilité** par article (avec période de battement).
- **Sélecteur de produits avancé** lors de la création de location (recherche, filtres, vignettes, disponibilité en temps réel).
- **Images cliquables** (lightbox) et **export PDF** des factures.

10. Description détaillée des modules

10.1 Matériel (inventaire)

Catalogue avec image, catégorie, marque, unité, devise, prix par jour, capacité, statut et **délai d'indisponibilité après location** (nettoyage, contrôle). Calendrier mensuel de disponibilité par article.

10.2 Locations

Création multi-articles avec **tarification au jour** (prix × quantité × durée), **TVA** automatique (HT → TVA → TTC), **paiements partiels** (suivi payé / reste), statuts (*en attente*, *en cours*, *retournée*, *annulée*) et **facture PDF**. La disponibilité tient compte des réservations concurrentes et du battement.

10.3 Approvisionnement (achats)

Gestion des **fournisseurs** et des **achats**. Un achat au statut *reçu* **alimente automatiquement** le stock. Suppression sécurisée (réajustement du stock).

10.4 Ventes

Vente de matériel avec TVA ; chaque vente **retire** les quantités du stock. La suppression d'une vente **restitue** le stock.

10.5 Dépenses

Saisie des charges par **catégorie** (loyer, salaires, électricité, carburant...), montant, date, fournisseur et mode de paiement.

10.6 Ajustements de stock

Entrées / sorties manuelles avec **motif** obligatoire (casse, perte, correction) et **valorisation** automatique au dernier prix d'achat. Historique horodaté et tracé par utilisateur.

10.7 Rapports

- **Bénéfice** : Revenus (locations + ventes) – Coûts (achats + dépenses) = Bénéfice net + marge %.
- **Locations** : nombre, chiffre d'affaires TTC, encaissé, reste à encaisser, répartition par statut et matériel le plus loué, le tout filtrable par période.

10.8 Paramètres

Page unique à onglets : **TVA, devises, unités, catégories, marques, types de paiement**. Une seule TVA et une seule devise « par défaut » par espace.

11. Adaptation au contexte marocain

- **Devise par défaut : Dirham (MAD / DH)** ; multi-devises avec taux de change.
- **TVA configurable** avec les taux marocains (20 %, 14 %, 10 %, 0 %).
- **Validation des numéros de téléphone** au format marocain (0X... / +212...).
- **Format des dates JJ/MM/AAAA** et fuseau **Africa/Casablanca**.
- **Interface bilingue français / arabe** avec mise en page **RTL**.

12. Déploiement et installation

```
# Backend (API Laravel)
cd backend
composer install
cp .env.example .env          # configurer la base MySQL
php artisan key:generate
php artisan migrate --seed
php artisan storage:link
php artisan serve --host=0.0.0.0 --port=8000

# Frontend (React)
cd frontend
npm install                   # configurer VITE_API_URL dans .env
npm run dev -- --host
```

L'application est accessible localement et sur le réseau local (LAN). Les pare-feux doivent autoriser les ports **8000** (API) et **5173** (frontend).

13. Comptes de démonstration

Rôle	Email	Mot de passe
Super-admin	admin@touslocations.com	1234567890
Manager (espace A)	manager@touslocations.com	1234567890
Manager (espace B)	manager2@touslocations.com	1234567890
Employé	employe@touslocations.com	1234567890
Client	client@touslocations.com	1234567890

Les données de démonstration incluent un catalogue avec images, des locations (plusieurs statuts), des ventes, des achats, des dépenses et des paiements — afin d'alimenter les rapports.

14. Perspectives d'évolution

- Paiement en ligne (Stripe / CMI) ;
 - Graphiques d'évolution (revenus / bénéfice par mois) ;
 - Export des rapports en PDF / Excel ;
 - Notifications automatiques (rappels de retour par e-mail / SMS) ;
 - Application mobile native ;
 - Branding par espace (logo, ICE sur les factures).
-

15. Conclusion

TousLocation couvre l'intégralité du cycle de gestion d'une entreprise de location de matériel, de l'approvisionnement jusqu'au reporting financier, dans une architecture moderne, sécurisée et multi-entreprises. L'application est **fonctionnelle, testée et déployée** en environnement local, et conçue pour évoluer (paiements en ligne, mobile, analyses avancées).

Diagrammes : voir [database-schema.html](#) (interactif, export PNG) et les fichiers sources `*.mmd` ainsi que les images du dossier [images/](#).